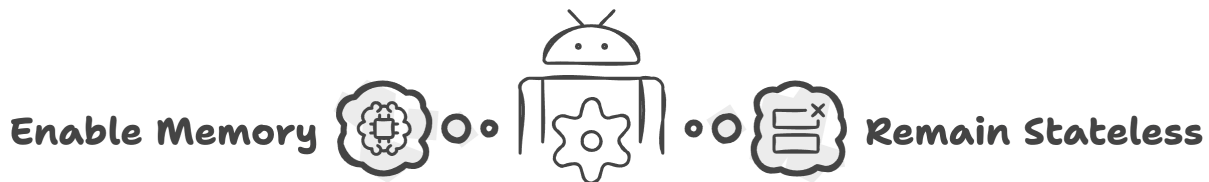


Maintaining State Across Agent Runs

Short-term memory can already accomplish a variety of tasks, but remembering past conversations is key to building truly context-aware agents. We'll see how to pass context forward so your agent doesn't greet us like a stranger each time.



Before continuing, let's run the code we've developed so far by running the hidden cells below:

! Note: Run the hidden cells below before running the rest of the code. !

3 hidden cells

By default, the `AgentWorkflow` is stateless between runs. This means that the agent will not have any memory of previous runs.

To maintain state, we need to keep track of the previous state. In LlamaIndex, Workflows have a `Context` that can be used to maintain state within and between runs. Since the `AgentWorkflow` is just a pre-built `Workflow`, we can also use it now.

To maintain state between runs, we'll create a new `Context` called `ctx`. We pass in our `workflow` to properly configure this Context object for the workflow that will use it.

With our configured Context, we can pass it to our first run.

```
from llama_index.core.workflow import (
    Context,
    Workflow,
    step,
    StartEvent,
    StopEvent,
)

class MyWorkflow(Workflow):
    @step()
    async def start(self, event: StartEvent) -> StopEvent:
        # Arguments passed to workflow.run() arrive in StartEvent.
        user_msg = event.get("user_msg", "No message provided.")

        # StopEvent ends the workflow and carries its final result.
        return StopEvent(result=f"Hello! You said: {user_msg}")

workflow = MyWorkflow()

# Optional for this example; useful for sharing or retaining state.
ctx = Context(workflow)

response = await workflow.run(
    user_msg="My name is Laurie, nice to meet you!",
    ctx=ctx,
)

print(response)
```

Hello! You said: My name is Laurie, nice to meet you!

Now we can pass the same context to a second run, and it will remember what happened before:

```
# Run the workflow again with the same context
response = await workflow.run(user_msg="What is my name?", ctx=ctx)
print(str(response))
```

Hello! You said: What is my name?

It's possible to maintain state over longer periods than a single run by serializing it to disk and deserializing it later when needed; check out [this tutorial](#)  for more insights.



Accessing Context from within tools

Sometimes our tools need an outlook on the wider world. Let's see how a tool can inspect the running context so its output remains focused on the current task.

By defining our tool to have access to the workflow context, we can set and retrieve variables from the context and use them in the tool or between tools.

`AgentWorkflow` uses a context variable called `state` that gets passed to every agent. We can rely on information in `state` being available without explicitly having to pass it in.

Note: To access the `Context`, the `Context` parameter should be the first parameter of the tool.

```

from llama_index.core.workflow import Context
from llama_index.core.agent.workflow import AgentWorkflow

# Prerequisite: llm must be an actual configured LlamaIndex LLM,
# not the DummyLLM instance from your example.

|
async def set_name(ctx: Context, name: str) -> str:
    """Save the user's name in the agent's shared state."""

    # Lock the state while updating it.
    async with ctx.store.edit_state() as ctx_state:
        ctx_state["state"]["name"] = name

    return f"Name set to {name}"

stateful_workflow = AgentWorkflow.from_tools_or_functions(
    [set_name],
    llm=llm,
    system_prompt=(
        "You are a helpful assistant. "
        "Whenever the user provides their name, call set_name to save it."
    ),
    # Start unset so you can observe the update.
    initial_state={"name": "unset"},
)

stateful_workflow_context = Context(stateful_workflow)

response = await stateful_workflow.run(
    user_msg="My name is Laurie",
    ctx=stateful_workflow_context,
)

print(str(response))

# Inspect the stored value to confirm the tool updated it.
state = await stateful_workflow_context.store.get("state")
print("Stored name:", state["name"])

```

Your name has been set to Laurie. How can I assist you today?
 Stored name: Laurie